

LBRO Interview Preparation Handbook

Complete Technical Guide for Placements, Viva, and Project Reviews

Project: LBRO — Law-aware Breach Response Orchestrator

Version: 2.0.0

Stack: FastAPI · PostgreSQL · React 18 · Gaussian Naive Bayes · Docker · AWS

SECTION 1: PROJECT OVERVIEW

1.1 What is LBRO?

LBRO (Law-aware Breach Response Orchestrator) is a full-stack cybersecurity platform that automates the entire incident response lifecycle — from detecting a security threat to generating a legally compliant breach notification.

Think of it as the "911 dispatcher + forensic lab + legal department" for cybersecurity, all running as a single deployable application.

Simple analogy: When your house alarm goes off:

1. The alarm system detects the intrusion (ML classification)
2. Security records who entered and when (Evidence Vault + Chain of Custody)
3. Police are notified automatically (Compliance Engine → GDPR/HIPAA deadlines)
4. A report is filed (PDF Report Generation)
5. Only authorized people can access the footage (RBAC)

LBRO does all of this for software systems.

1.2 Why Was It Built?

The problem: Cybersecurity incident response is broken for most organizations:

1. Tools are fragmented — separate SIEM, SOAR, forensic, and legal tools that don't talk to each other
2. Legal compliance is manual — nobody tells you which law applies to your breach or when the deadline is
3. Evidence handling is sloppy — files can be tampered with, access isn't logged, chain of custody breaks
4. Too expensive — Splunk costs \$50,000+/year; most SMEs can't afford it

5. Requires specialists — most SIEM tools need certified professionals to operate

LBRO's solution: One platform that integrates ML detection, evidence preservation, compliance automation, and reporting — deployable by a developer with `docker compose up`.

1.3 What Real-World Problem Does It Solve?

Scenario: You run an e-commerce startup. A hacker exfiltrates 10,000 customer records.

Without LBRO:

- You discover it 3 weeks later (too slow)
- You don't know GDPR requires notifying the EU Data Protection Authority within 72 hours
- Your firewall logs are unstructured; no chain of custody
- You spend \$50,000 on forensic consultants
- You miss the GDPR deadline and receive a €200,000 fine

With LBRO:

- The intrusion is detected automatically via ML (sub-millisecond classification)
- An incident is created with severity=critical
- GDPR obligations are automatically generated with a 72-hour countdown
- Evidence is stored with SHA-256 hash and full chain of custody
- A PDF forensic report is generated for legal proceedings
- Your analyst investigates using the 7-tab investigation workspace

1.4 Who Are the Target Users?

User Type	How They Use LBRO
Security Analyst	Investigates incidents, reviews ML classifications, uploads evidence
Project Admin	Creates projects, manages API keys, assigns roles
DevOps/Developer	Integrates the SDK into their application (3 lines of code)
Compliance Officer	Monitors GDPR/HIPAA/DPDPA obligations and deadlines
Platform Admin (super_admin)	Manages the entire LBRO deployment
SME Owner	Gets automated compliance and reports without a security team

1.5 How Does LBRO Differ from Existing SIEM/SOAR Platforms?

Feature	LBRO	Splunk	Wazuh	CrowdStrike
ML Attack Classification	✓	Partial	–	✓
Evidence Chain of Custody	✓	–	–	–
GDPR Compliance Engine	✓	Partial	Partial	–
HIPAA Compliance Engine	✓	✓	✓	–
DPDPA Compliance Engine	✓	–	–	–
Project API Key Ingestion	✓	–	–	–
PDF Report Generation	✓	✓	–	✓
Self-hosted / Open	✓	–	✓	–
Cost	Free	\$50k+/yr	Free	\$50k+/yr
Setup complexity	Low	High	High	High

- Key differentiators:
- 1. **Law-aware** — GDPR, HIPAA, DPDPA obligations auto-generated from incident metadata
 - 2. **Evidence-grade forensics** — SHA-256 hashing, immutability, full chain of custody
 - 3. **3-line integration** — any application can send events via `POST /api/v1/events`
 - 4. **Sub-millisecond ML** — Gaussian Naive Bayes classifies attacks in <1ms
 - 5. **All-in-one** — no need for separate SIEM + SOAR + forensic + compliance tools

1.6 Product Story (Complete Narrative)

LBRO was built as a specialization project to solve a real engineering problem: the gap between "we detected something suspicious" and "we've fulfilled our legal obligations." The project required deep integration of five domains: machine learning, web security, digital forensics, legal compliance, and cloud infrastructure.

The name "Law-aware Breach Response Orchestrator" reflects the core innovation: the system doesn't just detect and respond — it *understands legal context*. When an incident is created with `personal_data_involved=True` and `affected_jurisdictions=["EU"]`, LBRO automatically generates GDPR Article 33 notification obligations with a 72-hour countdown. No human needs to look up the law.

SECTION 2: COMPLETE ARCHITECTURE

2.1 Five-Tier Architecture Overview



[illegible]

2.2 Complete Component Map

Frontend Components

- **React 18.2 SPA** — Single Page Application, no server-side rendering
- **Zustand store** — global auth state, project selection
- **TanStack Query v5** — server state (caching, refetching, mutations)
- **React Router v6** — client-side routing with protected routes
- **Radix UI** — accessible primitive components (dialogs, dropdowns, tabs)
- **Tailwind CSS** — utility-first styling
- **Recharts** — dashboard charts (incident trends, severity distribution)
- **Axios** — HTTP client with interceptors for token refresh
- **Vite 5** — build tool and dev server

Backend Components

- **FastAPI 0.115.5** — ASGI framework, 16 routers
- **Uvicorn** — ASGI server (production: gunicorn + uvicorn workers)
- **SQLAlchemy 2.0 async** — ORM with `async_sessionmaker`
- **asyncpg 0.30.0** — async PostgreSQL driver
- **Alembic 1.14.0** — database migrations (11 migration files)

- **python-jose** — JWT creation and verification (HS256)
- **passlib + bcrypt 3.2.2** — password hashing
- **scikit-learn 1.5.2** — ML training and inference
- **reportlab 4.2.5** — PDF report generation
- **structlog 26.1.0** — structured JSON logging
- **pydantic 2.10.3** — data validation and settings

Infrastructure Components

- **Docker Compose** — 7 services: postgres, localstack, migrate, api, worker, frontend, seed
- **LocalStack** — AWS emulation (S3, SQS, SecretsManager) for local dev
- **Terraform** — IaC for AWS (ECS Fargate, RDS, S3, SQS, WAF, VPC)
- **PostgreSQL 16-alpine** — primary database

2.3 Event Flow: From Log Entry to Dashboard

Here is what happens when an external application sends a security event to LBRO:

Step 1 — External Application Sends Event

```
POST /api/v1/events
Authorization: Bearer proj_abc123...
Content-Type: application/json

{
  "event_type": "sql_injection",
  "severity": "critical",
  "source_ip": "185.220.101.42",
  "message": "SQL injection attempt in /api/users",
  "network_features": { "destination_port": 443, "syn_flag_count": 5 }
}
```

Step 2 — Middleware Stack Processes Request

1. **SecurityHeadersMiddleware** — adds security headers to response (runs after)
2. **RateLimitMiddleware** — checks sliding window counter for this IP+path
3. **TrustedHostMiddleware** — validates Host header
4. **CORSMiddleware** — validates origin header
5. **request_context_middleware** — assigns X-Request-ID, starts timer

Step 3 — FastAPI Router (events.py)

1. `get_project_from_api_key()` dependency runs
2. Extracts Bearer token, verifies `token.startswith("proj_")`
3. Queries `SELECT * FROM projects WHERE api_key = ? AND status = 'active'`

4. Returns `Project` object — `project_id` is now known server-side

Step 4 — Schema Validation (Pydantic)

- `SecurityEventCreate` schema validates the request body
- Invalid fields raise HTTP 422 Unprocessable Entity automatically

Step 5 — SecurityEvent Persisted

```
INSERT INTO security_events (id, project_id, event_type, severity, source_ip, message, processing_status)
VALUES (uuid(), ?, ?, ?, ?, ?, 'pending')
```

Step 6 — ML Classification

1. `AttackClassifier.predict()` called with network features
2. Features converted to 77-dimension numpy array
3. Sparse input guard: if fewer than 10 non-zero features → heuristic fallback
4. `StandardScaler` transforms feature vector
5. `GaussianNB` predicts probabilities for all 15 classes
6. `argmax` gives predicted class; confidence = max probability
7. If confidence < 0.75: `needs_review = True`
8. `SecurityEvent` updated: `ml_attack_category`, `ml_confidence`, `ml_model_version`

Step 7 — Incident Auto-Creation

- If `severity in ("critical", "high")`: new Incident created automatically
- `incident_id` written back to `SecurityEvent`

Step 8 — SSE Broadcast

- `_bus_publish(project_id, event_data)` pushes to in-memory pub/sub
- All connected frontend clients for this project receive the event via SSE

Step 9 — Compliance Check (on Incident creation)

- `ComplianceService.generate_obligations(incident)` called
- Checks `incident.personal_data_involved`, `incident.health_data_involved`, `incident.affected_jurisdictions`
- Generates GDPR/HIPAA/DPDPA obligation records with deadlines

Step 10 — Response to Client

```
{
  "id": "uuid",
  "event_type": "sql_injection",
  "ml_attack_category": "Web Attack - Sql Injection",
  "ml_confidence": 0.9842,
  "incident_id": "uuid",
  "processing_status": "processed"
}
```

Step 11 — Dashboard Updates

- Frontend React Query cache is invalidated via the SSE event
- Dashboard re-fetches incident counts, severity charts, recent events

2.4 Authentication Flow Sequence

Client	FastAPI	Database
--- POST /auth/login-->		
{email, password}		
	-- SELECT User -----	
	<--- User row -----	
	verify bcrypt hash	
	check lockout	
	check is_active	
	-- UPDATE last_login	
	reset failures	
<--- 200 OK -----		
{access_token,		
refresh_token}		
	(30 min later)	
--- POST /auth/refresh ->		
{refresh_token}		
	verify signature	
	check type=refresh	
<--- 200 OK -----		
{new_access_token,		
new_refresh_token}		
--- POST /auth/logout ->		
Bearer: access_token		
	decode jti	
	-- INSERT revoked_	
	tokens ----->	
<--- 204 No Content ----		

2.5 RBAC Authorization Flow

```

HTTP Request
  ■
  ▼
get_current_user()    ← checks Bearer JWT or X-API-Key
  ■
  ▼ User object
get_current_active_user() ← checks user.is_active
  ■
  ▼ User object

```

```

require_permission(P)      ← checks ROLE_PERMISSIONS[user.role]
  ■
  ■■■■■ is_super_admin(role)? ■■■■ YES → audit log + PASS
  ■
  ■■■■■ Role(user.role) valid? ■■■■ NO → 403 + audit log
  ■
  ■■■■■ permission in ROLE_PERMISSIONS[role]? ■■■■ NO → 403 + audit log
                                     YES → PASS → handler()

```

2.6 Database Schema (All 14 Tables)

```

users
  id (UUID PK)
  email (UNIQUE)
  username (UNIQUE)
  hashed_password
  role (viewer/analyst/admin/super_admin)
  failed_login_attempts
  locked_until
  api_key

projects
  id (UUID PK)
  name
  slug (UNIQUE)
  owner_id (FK → users)
  api_key (proj_<token>, UNIQUE)
  environment (development/staging/production)
  status (active/archived)

incidents
  id (UUID PK)
  project_id (FK → projects)
  title, description, status, severity
  attack_category, confidence_score, ml_model_version
  source_ip, destination_ip, destination_port
  network_features (JSON)
  affected_jurisdictions (JSON array)
  personal_data_involved, health_data_involved
  detected_at, created_at, updated_at

security_events
  id (UUID PK)
  project_id (FK → projects, resolved from API key)
  event_type, severity
  source_ip, message
  payload (JSON)
  ml_attack_category, ml_confidence
  incident_id (FK → incidents, nullable)
  processing_status (pending/processed/failed)

evidence
  id (UUID PK)
  incident_id (FK → incidents CASCADE)

```

```
filename, original_filename, content_type, file_size
sha256_hash (64 chars)
file_data (LargeBinary, DEFERRED – not loaded on list queries)
is_immutable (default True)
uploaded_by (FK → users)

chain_of_custody
  id (UUID PK)
  evidence_id (FK → evidence CASCADE)
  action (uploaded/accessed/exported/verified)
  performed_by (FK → users)
  performed_by_name, ip_address, hash_at_time
  created_at

compliance_records
  id (UUID PK)
  incident_id (FK → incidents)
  regulation (GDPR/HIPAA/DPDPA)
  jurisdiction
  obligation (text)
  deadline (datetime)
  is_met (bool)
  met_at

compliance_obligations
  id (UUID PK)
  project_id (FK → projects)
  framework, control_id, control_name
  status (not_started/in_progress/compliant/non_compliant)
  score (0-100)

compliance_assessments
  id (UUID PK)
  project_id (FK → projects)
  framework, overall_score, total_controls
  assessment_date

investigation_notes
  id (UUID PK)
  incident_id (FK → incidents)
  content, author_id (FK → users)

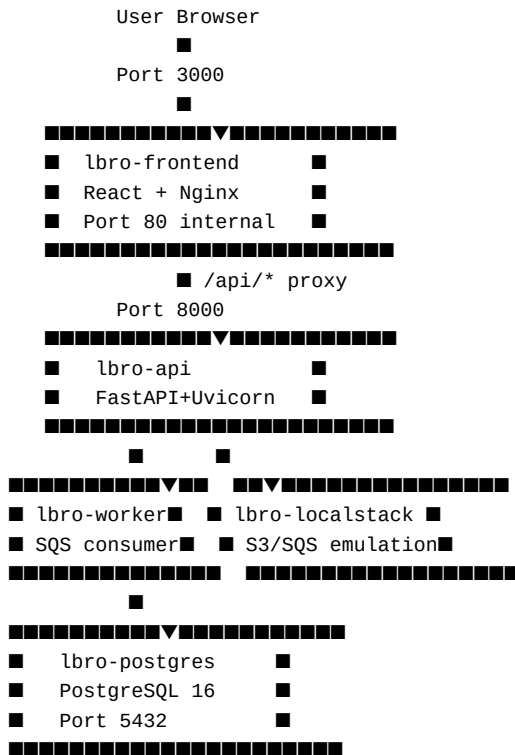
audit_logs
  id (UUID PK)
  user_id (FK → users), user_email
  action, resource_type, resource_id
  ip_address, user_agent
  request_method, request_path
  response_status, details (JSON)

revoked_tokens
  id (UUID PK)
  jti (UNIQUE – JWT ID claim)
  expires_at

notifications
  id (UUID PK)
```

```
incident_id (FK → incidents)
type, status, message
```

2.7 Deployment Architecture (Docker Compose)



Service startup order:

postgres (healthy) → localstack (healthy) → migrate (completed) → api + worker + seed

SECTION 3: TECH STACK — DEEP DIVE

3.1 React 18.2

What it is: A JavaScript library for building user interfaces using a component tree model.

Why we chose it:

- Concurrent features (Suspense, transitions) enable responsive UIs during data fetching
- Massive ecosystem — Radix UI, Recharts, React Query all integrate natively
- Component model maps perfectly to LBRO's 7-tab investigation workspace

- React 18's automatic batching reduces unnecessary re-renders

How React works internally:

1. Components are functions returning JSX
2. JSX is transpiled by Vite/esbuild to `React.createElement()` calls
3. React builds a Virtual DOM tree
4. On state change, React reconciles old and new Virtual DOM trees
5. Only changed DOM nodes are actually updated (diffing algorithm)
6. React Fiber (since v16) breaks reconciliation into small chunks to avoid blocking the main thread

Alternatives considered: Vue.js, Angular, Svelte

- Vue: smaller ecosystem for enterprise-grade component libraries
- Angular: TypeScript-first but too opinionated; heavier bundle
- Svelte: no runtime, but immature tooling for large apps

When NOT to use React: Static content sites (use Astro/Hugo), server-rendered apps where SEO is critical without SSR setup, simple forms (overkill)

3.2 TypeScript 5.3

What it is: A superset of JavaScript that adds static types, compiled to plain JS.

Why we chose it:

- Catches bugs at compile time: wrong API response shape, missing fields, incorrect function signatures
- IDE autocomplete for all API types — self-documenting code
- Large team codebases stay maintainable; types are living documentation
- React + TypeScript is the industry standard combination

How TypeScript works internally:

1. `.tsx` files parsed by TypeScript compiler (tsc)
2. Type checker analyzes types at compile time (no runtime overhead)
3. Output is plain JavaScript — types are erased at runtime
4. In LBRO, Vite uses esbuild for fast transpilation (skips type checking in dev, runs tsc separately for CI)

Key TypeScript features used in LBRO:

- Interface types for API responses (`IncidentResponse`, `UserResponse`)
- Generic types for TanStack Query hooks (`useQuery<Incident[]>`)
- Union types for status fields (`"pending" | "triaging" | "closed"`)
- `Annotated` equivalent via TypeScript generics

3.3 Vite 5

What it is: A modern frontend build tool that uses native ES modules in development and Rollup for production builds.

Why we chose it over Create React App (CRA):

- CRA uses Webpack which bundles everything; Vite uses native ES modules so only requested files are served — dev server starts in <300ms
- Hot Module Replacement (HMR) is instantaneous in Vite; CRA takes seconds
- Production builds use Rollup (faster than Webpack)
- Tree-shaking removes unused code automatically

How Vite works internally:

1. In dev: browser makes native ES module requests; Vite serves and transforms on-demand
2. No bundling in dev — each file transformed individually
3. In production: `vite build` runs Rollup, creates optimized bundle
4. Code splitting — separate chunks for routes, loaded lazily

3.4 Tailwind CSS

What it is: A utility-first CSS framework where you compose styles using pre-defined class names.

Why we chose it:

- No CSS context-switching — styles live in the component file
- No naming conflicts — no `.card` vs `.Card` vs `.card-container` debates
- Consistent design tokens — spacing, colors, typography all follow a system
- Works perfectly with Radix UI primitives (Radix handles behavior; Tailwind handles appearance)

How Tailwind works internally:

1. Scans all source files for class names used
2. Generates a CSS file containing only those classes (PurgeCSS/JIT engine)
3. Final CSS bundle is typically 5-15KB instead of full Tailwind's 3MB

3.5 FastAPI 0.115.5

What it is: A modern Python web framework for building APIs with automatic OpenAPI docs, async support, and Pydantic-based validation.

Why we chose it over Flask/Django:

Feature	FastAPI	Flask	Django
Async native	✓	Partial	Partial
Auto OpenAPI/Swagger	✓	Plugin	Plugin
Pydantic validation	✓ built-in	Manual	Manual
Type hints	First-class	Optional	Optional
Performance	Very high (Starlette)	Medium	Lower
Learning curve	Low	Low	Medium

How FastAPI works internally:

1. Built on **Starlette** (ASGI framework) and **Pydantic** (validation)
2. Uses Python type hints to generate request/response schemas at startup
3. Dependency injection system (`Depends()`) resolves nested dependencies automatically
4. At each request: Starlette routes → Pydantic validation → your handler
5. OpenAPI schema is auto-generated from function signatures and Pydantic models
6. `async def` handlers run in the asyncio event loop; `def` handlers run in a thread pool

Pydantic integration:

- Request bodies are validated automatically via Pydantic models
- Invalid data raises `RequestValidationError` → HTTP 422 automatically
- Response models (`response_model=...`) serialize and validate output

3.6 SQLAlchemy 2.0 Async

What it is: Python's most popular ORM (Object-Relational Mapper) — maps Python classes to database tables.

Why async?

- FastAPI is async; database I/O is the slowest operation
- Synchronous SQLAlchemy would block the event loop on every query
- `AsyncSession` + `asyncpg` allows thousands of concurrent requests without spawning threads

Key concepts used in LBRO:

```
# Session factory – configured once at startup
AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,  # Objects don't expire after commit
    autoflush=False,        # Manual control over when writes go to DB
    autocommit=False,        # Explicit commit required
)

# Dependency – yields session to each request
```

```

async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()      # Auto-commit on success
        except Exception:
            await session.rollback()    # Auto-rollback on error
            raise

```

flush() vs commit():

- `db.flush()` — writes to the transaction buffer (visible within same transaction, NOT yet in DB)
- `db.commit()` — persists to database permanently and releases transaction
- Critical bug we fixed: demo data functions were calling `flush()` but not `commit()`, so all data was silently rolled back when the session closed

Deferred loading (Evidence.file_data):

```

# file_data is NOT loaded on regular queries (avoids 100MB in memory)
file_data: Mapped[bytes | None] = deferred(mapped_column(LargeBinary, nullable=True))

# Only loaded when explicitly requested:
result = await db.execute(
    select(Evidence).options(undefer(Evidence.file_data))
)

```

3.7 asyncpg 0.30.0

What it is: A high-performance PostgreSQL driver for asyncio. Written in Cython for speed.

Why asyncpg over psycopg2:

- psycopg2 is synchronous — blocks the event loop
- asyncpg is pure async — 3-5x faster than psycopg2 for concurrent workloads
- LBRO uses `postgresql+asyncpg://` in the DATABASE_URL

Connection pooling: SQLAlchemy manages a pool of 10 connections (DATABASE_POOL_SIZE) with 20 overflow. The pool pre-pings connections to detect stale ones.

3.8 PostgreSQL 16

What it is: Enterprise-grade open-source relational database.

Why PostgreSQL over MongoDB:

Requirement	PostgreSQL	MongoDB
Referential integrity	FK constraints, CASCADE	No native FK

Requirement	PostgreSQL	MongoDB
ACID transactions	Full ACID	Multi-document (v4+)
Complex joins	Native SQL joins	Aggregation pipeline
JSON support	JSONB — indexed, queryable	Native JSON
Forensic chain of custody	FK + CASCADE guarantees	Manual enforcement
UUID primary keys	Native UUID type	ObjectId

Why PostgreSQL fits LBRO perfectly:

- Chain of custody requires guaranteed referential integrity — can't delete evidence without cascading
- Compliance records join through incidents to projects — complex joins needed
- Audit logs must be append-only — SQL enforces this via row-level triggers if needed
- The `JSONB` type stores `network_features` and `payload` as queryable JSON

3.9 Alembic 1.14.0

What it is: Database migration tool for SQLAlchemy — manages schema changes over time.

LBRO has 11 migration files:

- 001: initial schema (users, incidents, evidence, chain_of_custody)
- 002: compliance tables
- 003: audit_logs
- 004: notifications
- 005: projects table + project_id FK on incidents
- 006: investigation_notes
- 007: security_events table
- 008: revoked_tokens table
- 009: compliance_obligations + assessments
- 010: project_member model
- 011: security_event processing columns

How Alembic works:

1. `alembic revision --autogenerate -m "description"` — generates migration from SQLAlchemy model diff
2. Migration file has `upgrade()` and `downgrade()` functions
3. `alembic upgrade head` — runs all pending migrations
4. `alembic_version` table tracks which migration is current

In Docker Compose: The `migrate` service runs `python /app/scripts/run_migrations.py` (not `alembic upgrade head` directly, to avoid PYTHONPATH conflicts). API and worker wait for `migrate: condition:`

```
service_completed_successfully.
```

3.10 JWT (JSON Web Tokens)

What it is: A compact, URL-safe token format for representing claims between two parties.

JWT Structure: Three Base64-encoded sections separated by dots:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyLWlkIiwiaXNjaXQ4MDAwMDAwfQ.signature
```

LBRO JWT claims:

```
{
  "sub": "user-uuid",           // Subject – user ID
  "exp": 1638000000,           // Expiry – 30 minutes
  "type": "access",             // Token type (access vs refresh)
  "jti": "unique-uuid",         // JWT ID – for revocation
  "role": "analyst",            // User role (embedded for quick RBAC)
}
```

Why JWT over sessions:

- Sessions require server-side storage — doesn't scale horizontally
- JWT is stateless — any API server can validate without a DB lookup
- The `jwt` claim allows per-token revocation without full session store

Token revocation (jti blacklist):

```
# On logout: extract jti from token, store in DB
revoked = RevokedToken(jti=jti, expires_at=datetime.fromtimestamp(exp))
db.add(revoked)
await db.commit()

# On every authenticated request: check revocation
revoked = await db.execute(select(RevokedToken).where(RevokedToken.jti == jti))
if revoked:
    raise HTTP 401 "Token has been revoked"
```

Refresh token flow:

- Access token: 30 minutes (short — limits damage if stolen)
- Refresh token: 7 days (long — stored securely by client)
- Refresh token lacks `jti` — intentional, can't be individually revoked
- To revoke a refresh token: the user must change their password

3.11 Bcrypt / Passlib

What it is: A password hashing algorithm designed specifically for password storage.

Why bcrypt over MD5/SHA-256:

- MD5/SHA-256 are fast — GPU can compute billions per second → brute force possible
- bcrypt is deliberately slow — configurable cost factor
- bcrypt includes a random 128-bit salt — same password produces different hash every time
- Adaptive cost — can be made slower as hardware gets faster

LBRO implementation:

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password) # Generates salt + hashes

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed) # Constant-time comparison
```

Critical pinning in LBRO: `bcrypt==3.2.2` (NOT 4.x)

- bcrypt 4.x raises `ValueError` when passwords exceed 72 bytes
- passlib 1.7.4's `detect_wrap_bug()` passes >72-byte passwords to `hashpw()`
- Solution: pin to bcrypt 3.2.2 which silently truncates (expected, documented behavior)

Account lockout:

- 5 failed attempts triggers lockout (`MAX_LOGIN_ATTEMPTS=5`)
- Locked for 15 minutes (`LOCKOUT_DURATION_MINUTES=15`)
- `locked_until` field on User model; checked before bcrypt verification

3.12 Docker + Docker Compose

What Docker is: A containerization platform that packages an application with all its dependencies into an isolated, reproducible container.

Key concepts:

- **Image:** Immutable snapshot (like a class)
- **Container:** Running instance of an image (like an object)
- **Dockerfile:** Script that builds an image
- **Volume:** Persistent storage that survives container restarts
- **Network:** Virtual network connecting containers

LBRO's 7 Docker services:

Service	Purpose	Health Check
postgres	PostgreSQL 16 database	<code>`pg_isready`</code>
localstack	AWS S3/SQS emulation	HTTP <code>`/health`</code>
migrate	Runs Alembic once, exits	<code>`service_completed_successfully`</code>
api	FastAPI + Uvicorn	HTTP <code>`/health`</code>
worker	SQS message consumer	restart: unless-stopped
frontend	React + Nginx	HTTP <code>`/health`</code>
seed	Creates default admin	runs once, exits

Why Docker Compose over running locally:

- Guaranteed identical environment — "works on my machine" eliminated
- Service orchestration — `depends_on: condition: service_healthy` ensures correct startup order
- Volume persistence — postgres data survives container restarts
- Network isolation — services communicate by name (`postgres:5432`), not localhost

3.13 AWS Services (Production Deployment)

ECS Fargate: Serverless container orchestration — runs Docker containers without managing EC2 instances. LBRO's api and worker services run as Fargate tasks.

RDS PostgreSQL: Managed PostgreSQL with automated backups, Multi-AZ failover, and read replicas. Production replacement for the Docker postgres service.

S3: Object storage for evidence files and PDF reports. In local dev, LocalStack emulates S3 at `http://localstack:4566`.

SQS (Simple Queue Service): Message queue for async processing. The worker service polls SQS for incident processing tasks.

WAF (Web Application Firewall): Filters malicious HTTP traffic before it reaches ECS.

Secrets Manager: Stores SECRET_KEY, DATABASE_URL, and other secrets. Pulled at container startup.

Terraform IaC: LBRO includes `terraform/` directory with modules for all AWS resources. `terraform apply` provisions the entire production infrastructure.

3.14 Scikit-learn + Gaussian Naive Bayes

Scikit-learn: Python's ML library — consistent API for 50+ algorithms.

Gaussian Naive Bayes — How It Works:

The "Naive" means it assumes all features are independent (which is mathematically incorrect for network flows, but works well in practice).

For classification, it computes:

$$P(\text{class} \mid \text{features}) \propto P(\text{class}) \times \prod P(\text{feature}_i \mid \text{class})$$

For continuous features, it assumes a Gaussian (normal) distribution:

$$P(\text{feature}_i \mid \text{class}) = (1/\sqrt{2\pi\sigma^2}) \times \exp(-(x-\mu)^2/2\sigma^2)$$

Where μ and σ are learned from the training data for each feature/class combination.

var_smoothing=1e-12: Adds a small value to variance to prevent numerical underflow when a feature has very low variance in a class.

Why GNB won over Decision Tree (which had higher raw F1):

- Composite score optimizes for multiple metrics simultaneously
- After tuning, GNB scored 0.9731 composite vs DT's 0.9674
- GNB's balanced accuracy (0.9678) beats DT's (0.9600) — better at rare classes
- GNB: <0.1s training, <1ms inference, 19KB model vs DT: larger model, slower on some inputs

SECTION 4: WHY DID WE CHOOSE THIS STACK?

4.1 Why FastAPI Instead of Flask?

Flask is fine for small projects but LBRO needed:

1. **Native async** — LBRO has 16 endpoints with concurrent database queries. Flask's synchronous model would require threading or gevent hacks.
2. **Automatic OpenAPI** — FastAPI generates Swagger UI from type hints; Flask needs separate libraries (Flask-RESTX, Flasgger)
3. **Pydantic validation** — built into FastAPI; Flask needs Marshmallow or WTForms
4. **Type safety** — FastAPI's dependency injection is type-checked by mypy; Flask's `g` and `request` context vars are not
5. **Performance** — FastAPI's Starlette foundation processes requests ~3x faster than Flask

Why not Django?

- Django REST Framework is synchronous by default

- Django's ORM doesn't support async natively until Django 4.1 (experimental)
- Django includes features we don't need: admin panel, sessions, templates
- Django's ORM is less flexible than SQLAlchemy for complex queries

4.2 Why PostgreSQL Instead of MongoDB?

LBRO's data model is inherently relational:

- Evidence belongs to an Incident (foreign key + CASCADE delete)
- ChainOfCustody belongs to Evidence (foreign key + CASCADE delete)
- ComplianceRecords join through Incidents to Projects

MongoDB would cause real problems:

- No foreign key enforcement — could create orphaned evidence records
- No transactions across collections — chain of custody could be partially written
- No joins — would require multiple round-trips or \$lookup aggregations
- Evidence integrity (SHA-256 verification) requires ACID guarantees

The JSON advantage: PostgreSQL's `JSONB` type handles our dynamic `network_features` and `payload` fields with indexing — best of both worlds.

4.3 Why JWT Instead of Sessions?

Aspect	JWT (LBRO)	Sessions
Storage	Client stores token	Server stores session
Scalability	Stateless — works across multiple servers	Requires shared session store (Redis)
Revocation	jti blacklist	Simple — delete session row
Expiry	Built into token	Configurable
Microservices	Token validates at any service	Session store must be shared

LBRO's choice: JWT with `jti` blacklist gives us stateless scalability while still supporting logout. The blacklist only needs to hold tokens until they expire naturally (30 minutes for access tokens).

4.4 Why Docker Instead of Running Locally?

Developer experience problem without Docker:

- Developer A: Python 3.10, PostgreSQL 14 → works
- Developer B: Python 3.12, PostgreSQL 16 → different behavior
- CI/CD: Python 3.11, PostgreSQL 15 → different again

With Docker:

- Same container image everywhere
- `docker compose up --build` — entire stack starts in one command
- LocalStack provides AWS services locally — no real AWS account needed for development
- Reproducible — `docker compose down -v &&` `docker compose up` resets to known state

4.5 Why React Instead of Vue or Angular?

React:

- Largest ecosystem — Radix UI, Recharts, TanStack Query all have first-class React support
- Concurrent features in React 18 — Suspense boundaries for loading states
- React Query (TanStack) is the best server state management library, React-native

Vue: Smaller component ecosystem for enterprise-grade primitives (no Radix UI equivalent)

Angular: Enterprise-first; TypeScript required; heavier bundle; two-way binding philosophy clashes with React's unidirectional data flow

4.6 Why REST Instead of GraphQL?

LBRO uses REST for pragmatic reasons:

- **Complexity:** GraphQL requires a schema definition layer, resolvers, and type system in addition to the REST layer. FastAPI already generates OpenAPI automatically.
- **Over-fetching is solved:** LBRO's endpoints return well-defined response shapes; clients don't need arbitrary field selection
- **Tool compatibility:** Swagger/OpenAPI generated by FastAPI works with curl, Postman, and all HTTP clients without a GraphQL client
- **Learning curve:** GraphQL has a steeper learning curve; REST is universally understood

When GraphQL would make sense: If clients need to compose arbitrary queries across multiple resource types (e.g., a mobile app with very different data needs than the web app).

4.7 Why Project-Scoped API Keys Instead of User Tokens?

Problem: External applications (food ordering systems, college projects) need to send events to LBRO. Having them use user JWT tokens would:

- Expose user credentials in application code
- Require users to manage token refresh
- Give the external app user-level permissions (dangerous)

Solution: Project API Keys

- Format: `proj_<secrets.token_urlsafe(32)>` — cryptographically random, 43 characters
- Only valid for the ingestion endpoint (`/api/v1/events`)
- Project_id resolved server-side — the app never knows or specifies the project_id
- Rotatable from the LBRO dashboard without affecting user accounts
- Rejected by `get_current_user()` if used as a user token

SECTION 5: MACHINE LEARNING DEEP DIVE

5.1 The Dataset — CICIDS2017

CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) is the most widely used benchmark dataset for network intrusion detection research.

Key properties:

- Captured using CICFlowMeter — extracts 78 statistical features from raw pcap files
- Represents 5 days of realistic network traffic (Monday through Friday)
- Monday: only BENIGN traffic (establishes normal baseline)
- Tuesday–Friday: progressively more attack traffic

LBRO's subset:

- 15,000 samples drawn with stratified sampling (maintains class proportions)
- 80/20 train/test split → 12,000 training, 3,000 test
- 77 features (one feature dropped during initial cleanup)
- 15 classes (1 BENIGN + 14 attack types)

Class imbalance:

- BENIGN: ~51% of traffic (majority class)
- Rare classes: Heartbleed, Infiltration (<0.5% each)
- This is realistic — most network traffic is benign

5.2 The 77 Features — What They Measure

The features measure statistics about network flow (not individual packets):

Packet-length statistics (10 features):

- `fwd_packet_length_max/min/mean/std` — distribution of forward packet sizes
- `bwd_packet_length_max/min/mean/std` — distribution of backward packet sizes
- Large variance in packet size → potential data exfiltration

Flow-level statistics (6 features):

- `flow_bytes_per_sec` — bandwidth usage
- `flow_packets_per_sec` — packet rate
- `flow_duration` — how long the flow lasted
- DDoS: very high `flow_packets_per_sec`
- PortScan: very low `flow_packets_per_sec`, short `flow_duration`

TCP flag counts (10 features):

- `syn_flag_count` — SYN packets (connection requests)
- `fin_flag_count` — FIN packets (connection teardown)
- `rst_flag_count` — RST packets (connection reset)
- SYN flood: extremely high `syn_flag_count` with no ACKs
- Port scan: many SYN packets to different ports

Inter-arrival times (IAT — 10 features):

- Time between consecutive packets
- DoS attacks: very low IAT (packets sent as fast as possible)
- Human browsing: variable IAT

Bulk and subflow statistics:

- Measure data transfer patterns
- Bot traffic: regular bulk transfers at predictable intervals

5.3 Preprocessing Pipeline

```
# Step 1: Handle NaN and infinity
features_df.replace([np.inf, -np.inf], np.nan, inplace=True)
features_df.fillna(0, inplace=True)

# Step 2: Clip outliers at 99.9th percentile
for col in features_df.columns:
    upper = features_df[col].quantile(0.999)
    features_df[col] = features_df[col].clip(upper=upper)

# Step 3: Scale features (for scale-sensitive models)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
# Save scaler for inference: scaler.pkl

# Step 4: Encode labels
```

```

le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
# 'BENIGN' → 0, 'Bot' → 1, 'DDoS' → 2, etc.
# Save encoder: label_encoder.pkl

```

5.4 Model Training and Evaluation

The 9 models evaluated (Gradient Boosting excluded due to timeout):

Model	Notes
Logistic Regression	L2, class_weight='balanced', max_iter=1000
Decision Tree	class_weight='balanced', GINI criterion
Random Forest	100 estimators, class_weight='balanced'
Extra Trees	100 estimators, class_weight='balanced'
AdaBoost	50 estimators, lr=1.0
Gaussian Naive Bayes	No hyperparams initially
KNN	k=5, StandardScaler
Linear SVM	LinearSVC, class_weight='balanced', StandardScaler
MLP	(100,50) hidden layers, StandardScaler

5.5 Evaluation Metrics — Mathematical Definitions

Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Problem for LBRO: BENIGN is 51% of data. A model predicting BENIGN for everything gets 51% accuracy but catches zero attacks.

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

"Of everything I said was a DDoS attack, how many actually were?"

Recall (Sensitivity)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

"Of all actual DDoS attacks, how many did I catch?"

In cybersecurity, **recall is more important** — missing an attack (FN) is worse than a false alarm (FP).

F1 Score

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Harmonic mean of precision and recall. Better than accuracy when classes are imbalanced.

Macro F1

$$\text{Macro F1} = \text{average}(\text{F1_score per class})$$

Treats all 15 classes equally. A class with 10 samples counts as much as one with 7,000. Penalizes poor performance on rare classes like Heartbleed.

LBRO's GNB macro F1 = 0.9692 — very strong on rare attack classes.

Balanced Accuracy

$$\begin{aligned} \text{Balanced Accuracy} &= \text{average}(\text{Recall per class}) \\ &= (\text{Recall_BENIGN} + \text{Recall_DoS} + \dots + \text{Recall_Heartbleed}) / 15 \end{aligned}$$

Corrects for class imbalance. Not inflated by majority class.

LBRO's GNB balanced accuracy = 0.9678

Matthews Correlation Coefficient (MCC)

$$MCC = (TP \times TN - FP \times FN) / \sqrt{((TP + FP)(TP + FN)(TN + FP)(TN + FN))}$$

Ranges from -1 to +1. Considered the most informative single metric for binary and multiclass problems. Accounts for all four cells of the confusion matrix.

LBRO's GNB MCC = 0.9957 — essentially perfect correlation between predictions and truth.

Cross-Validation F1

- 3-fold stratified CV: data split into 3 parts, train on 2, test on 1, repeat 3 times
- **CV F1 = mean(F1 across 3 folds)** — measures generalization, not memorization
- **LBRO's GNB CV F1 = 0.9595** — stable, not overfit to training data

Composite Score (LBRO's custom metric)

$$\text{Composite} = F1_macro \times 0.4 + \text{Balanced_Accuracy} \times 0.3 + MCC \times 0.2 + CV_F1 \times 0.1$$

Weights explained:

- F1_macro (40%): primary metric — must perform well on all attack classes
- Balanced Accuracy (30%): must handle class imbalance well
- MCC (20%): overall correlation quality

- CV F1 (10%): generalization check — small weight because it uses fewer folds

5.6 The Sparse Input Guard

Problem discovered: When testing with events that only have 2-3 CICIDS2017 features (most log events), GaussianNB collapsed to predicting PortScan with confidence=1.0 for everything.

Root cause: GNB with very sparse inputs (mostly zeros) sees $P(x|\text{PortScan}) \approx 1.0$ because PortScan training samples have many zero features. The argmax picks PortScan.

Fix implemented:

```
MIN_FEATURES_FOR_MODEL = 10

non_zero_count = int(np.count_nonzero(vec))
if non_zero_count < MIN_FEATURES_FOR_MODEL:
    return self._heuristic_predict(features) # Fall back to rules
```

Heuristic fallback rules:

- `flow_packets_per_sec > 10000` → DDoS
- `syn_flag_count > 1000` → DoS Hulk
- `destination_port == 21` → FTP-Patator
- `destination_port == 22` → SSH-Patator
- `destination_port in (80, 443, 8080)` → Web Attack - Brute Force
- Otherwise → BENIGN
- Always returns confidence = 0.65 (below `ML_CONFIDENCE_THRESHOLD=0.75` → `needs_review=True`)

SECTION 6: DATABASE DESIGN

6.1 Why UUID Primary Keys?

LBRO uses `UUID(as_uuid=True)` for all primary keys instead of auto-increment integers.

Advantages:

1. **Security** — sequential IDs leak business metrics (`/incidents/1`, `/incidents/2` reveals count)
2. **Distributed systems** — UUIDs can be generated independently across services without coordination
3. **API safety** — clients cannot guess or enumerate resource IDs
4. **Merge operations** — no collision risk when combining data from multiple systems

Trade-off: UUIDs are 128-bit (16 bytes) vs integer (4-8 bytes) — slightly larger indexes. PostgreSQL's UUID type handles this efficiently.

6.2 Database Normalization

LBRO is in **3rd Normal Form (3NF)**:

- 1NF: Every attribute is atomic (no repeating groups)
- 2NF: No partial dependencies (all non-key attributes depend on entire PK)
- 3NF: No transitive dependencies (non-key attributes don't depend on other non-key attributes)

Example: ComplianceRecord stores `incident_id` (FK), not a copy of `incident.title`. The title is always fetched via join — no data duplication.

Controlled denormalization:

- `chain_of_custody.performed_by_name` — stores the name at time of access, because `users.full_name` can change. This is intentional forensic denormalization — chain of custody must record who exactly did what, even if they later changed their name.

6.3 Indexes

Explicitly indexed columns in LBRO:

- `incidents.project_id` — most queries filter by project
- `incidents.status` — dashboard filters by status
- `incidents.severity` — dashboard filters by severity
- `security_events.project_id` — live event stream filters by project
- `security_events.source_ip` — threat intelligence lookups
- `evidence.incident_id` — evidence list for an incident
- `chain_of_custody.evidence_id` — custody chain for an evidence item
- `projects.slug` — unique project slug lookup
- `projects.api_key` — API key authentication lookup (must be fast)
- `users.email` — login lookup
- `revoked_tokens.jti` — token revocation check (every request)

Why `revoked_tokens.jti` must be indexed: Every authenticated request checks this table. Without an index, this is a full table scan on every API call.

6.4 Foreign Key Behavior

`users`  `incidents.created_by` → SET NULL on user delete

```

users ████████████████████████████████████████ incidents.assigned_to → SET NULL on user delete
projects ████████████████████████████████████████ incidents.project_id → SET NULL on project delete
incidents ████████████████████████████████████████ evidence.incident_id → CASCADE DELETE
evidence ████████████████████████████████████████ chain_of_custody.evidence_id → CASCADE DELETE
incidents ████████████████████████████████████████ compliance_records.incident_id → (no explicit action)
projects ████████████████████████████████████████ security_events.project_id → CASCADE DELETE

```

CASCADE DELETE reasoning: When an incident is deleted, all its evidence and chain of custody records should be deleted too — no orphaned forensic data. When a project is deleted, all its security events go with it.

SET NULL reasoning: If a user is deleted, their incidents remain (for forensic/historical purposes) but the `assigned_to` field is nulled — the incident persists but is unassigned.

6.5 Alembic Migration Workflow

```

# Generate new migration from model changes
alembic revision --autogenerate -m "add threat_score to incidents"

# Apply all pending migrations
alembic upgrade head

# Roll back one migration
alembic downgrade -1

# Check current migration
alembic current

# View migration history
alembic history

```

In production (Docker): The `migrate` container runs on startup and exits. The `api` container's `depends_on: migrate: condition: service_completed_successfully` ensures no API server starts until migrations complete.

SECTION 7: AUTHENTICATION DEEP DIVE

7.1 Complete Login Flow

```

# 1. Client sends credentials
POST /api/v1/auth/login
{"email": "alice@lbro.io", "password": "SecurePass123!"}

# 2. AuthService.login() called
async def login(self, data: LoginRequest) -> TokenResponse:
    # a) Fetch user by email
    user = await db.execute(select(User).where(User.email == data.email))

```

```

# b) Check if account locked
if user.locked_until and user.locked_until > datetime.now(timezone.utc):
    raise HTTPException(403, "Account locked. Try in X minutes.")

# c) Verify bcrypt hash (deliberately slow: 100-200ms)
if not verify_password(data.password, user.hashed_password):
    user.failed_login_attempts += 1
    if user.failed_login_attempts >= 5:
        user.locked_until = now + timedelta(minutes=15)
    await db.commit()
    raise HTTPException(401, "Invalid credentials")

# d) Reset failure counter on success
user.failed_login_attempts = 0
user.last_login = datetime.now(timezone.utc)

# e) Create tokens
extra = {"role": user.role, "email": user.email}
access_token = create_access_token(subject=user.id, extra=extra)
refresh_token = create_refresh_token(subject=user.id)

return TokenResponse(
    access_token=access_token,
    refresh_token=refresh_token,
    token_type="bearer",
    expires_in=settings.ACCESS_TOKEN_EXPIRE_MINUTES * 60
)

```

7.2 Token Verification on Every Request

```

# Every protected endpoint calls this dependency chain:
get_current_user() → get_current_active_user() → require_permission(P)

# Inside get_current_user():
# 1. Extract Bearer token from Authorization header
# 2. Decode JWT with HS256 secret key
payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])

# 3. Verify token type
if payload.get("type") != "access":
    raise HTTP 401

# 4. Check JTI revocation (DB lookup)
jti = payload.get("jti")
revoked = await db.execute(select(RevokedToken).where(RevokedToken.jti == jti))
if revoked.scalar_one_or_none():
    raise HTTP 401 "Token has been revoked"

# 5. Load user from DB
user = await db.execute(select(User).where(User.id == user_id))
return user

```

7.3 API Key Authentication (Two Types)

Type 1: User API Keys (`lbro_<token>`)

- Format: `lbro_ + secrets.token_urlsafe(32)` = 47 characters
- Sent via `X-API-Key` header
- Used by automation tools, CI/CD pipelines
- Subject to same RBAC as the user

Type 2: Project API Keys (`proj_<token>`)

- Format: `proj_ + secrets.token_urlsafe(32)` = 47 characters
 - Sent via `Authorization: Bearer proj_...`
 - Used ONLY for event ingestion (`/api/v1/events`)
 - No user identity — resolves directly to a Project
 - Cannot be used for any other endpoint (rejected in `get_current_user()`)
-

SECTION 8: RBAC DEEP DIVE

8.1 The Four Roles

super_admin (platform level)

- Manages the entire LBRO deployment
- Holds ALL permissions — both project and platform
- Every action is audit-logged with `action="super_admin_access"`
- Can cross project boundaries (view incidents from any project)
- Used for: system health checks, platform-wide user management

admin (project level)

- Holds ALL project-level permissions
- Can create/delete incidents, manage users, rotate API keys, approve reports
- Cannot cross project boundaries (only sees their project's data)
- Used for: project setup, team management

analyst (project level)

- Holds project permissions minus: `DELETE_INCIDENT`, `DELETE_EVIDENCE`, `MANAGE_USERS`, `MANAGE_ROLES`, `ROTATE_API_KEYS`, `SYSTEM_SETTINGS`
- Can investigate incidents, upload evidence, generate reports

- Used for: day-to-day incident response

viewer (project level)

- Can only READ: incidents, evidence (download), dashboard, notifications, compliance, reports, ML info
- Cannot modify anything
- Used for: stakeholders, auditors, read-only observers

8.2 Permission Enforcement Flow

```
# In any protected endpoint:
@router.delete("/incidents/{incident_id}")
async def delete_incident(
    incident_id: uuid.UUID,
    # This dependency resolves and checks permission:
    current_user: Annotated[User, Depends(require_permission(Permission.DELETE_INCIDENT))],
    db: Annotated[AsyncSession, Depends(get_db)],
):
    # Only reached if permission check passes
    ...

# Inside require_permission(DELETE_INCIDENT):
# 1. Is user super_admin? → audit log + PASS
# 2. Is role valid? → if not, 403 + audit log
# 3. Does ROLE_PERMISSIONS[role] contain DELETE_INCIDENT? → if not, 403 + audit log
# 4. PASS → handler runs
```

8.3 The ROLE_PERMISSIONS Dictionary

```
ROLE_PERMISSIONS = {
    Role.VIEWER: {
        Permission.READ_INCIDENT,
        Permission.DOWNLOAD_EVIDENCE,
        Permission.VIEW_DASHBOARD,
        Permission.READ_NOTIFICATION,
        Permission.VIEW_COMPLIANCE,
        Permission.VIEW_REPORT,
        Permission.VIEW_ML,
    },
    Role.ANALYST: VIEWER_PERMISSIONS | {
        Permission.CREATE_INCIDENT,
        Permission.UPDATE_INCIDENT,
        Permission.ASSIGN_INCIDENT,
        Permission.UPLOAD_EVIDENCE,
        Permission.GENERATE_REPORT,
        Permission.APPROVE_REPORT,
        Permission.VIEW_AUDIT,
        Permission.EXPORT_AUDIT,
        Permission.APPROVE_NOTIFICATION,
        Permission.DISPATCH_NOTIFICATION,
        Permission.MANAGE_COMPLIANCE,
    },
}
```

```

        Permission.VIEW_INFRASTRUCTURE,
    },
    Role.ADMIN: set(Permission),      # All 30 permissions
    Role.SUPER_ADMIN: set(Permission), # All 30 permissions + platform bypass
}

```

Why not compare role strings?

```

# WRONG – brittle, hard to maintain
if user.role == "admin" or user.role == "super_admin":
    allow_delete()

# RIGHT – single source of truth
if has_permission(Role(user.role), Permission.DELETE_INCIDENT):
    allow_delete()

```

The first approach breaks whenever a role is added or renamed. The second approach is centralized — adding a new role only requires one dictionary entry.

SECTION 9: COMPLETE REQUEST FLOW — FAILED LOGIN EXAMPLE

Let's trace a failed login attempt step by step.

Request: `POST /api/v1/auth/login` with wrong password

1. Network → Nginx (Frontend Container)

- Browser sends HTTP POST to `http://localhost:3000/api/v1/auth/login`
- Nginx configuration in frontend container: `/api/*` → proxy to `http://api:8000`
- Nginx strips CORS headers from response (FastAPI handles CORS)

2. Nginx → FastAPI (api:8000)

- TCP connection established to Uvicorn worker
- Uvicorn passes ASGI scope to Starlette

3. Middleware Stack (in order, outermost first)

- `SecurityHeadersMiddleware.dispatch()` — called but defers to next (headers added to RESPONSE)
- `RateLimitMiddleware.dispatch()` — checks `{client_ip}:{/api/v1/auth/login}`

- If 10+ requests in 60 seconds → returns 429 immediately

- Otherwise, records timestamp, passes through

- `TrustedHostMiddleware` — validates Host header
- `CORSMiddleware` — validates Origin header for preflight

- `request_context_middleware` — assigns X-Request-ID, starts `time.perf_counter()`

4. Router Matching

- Starlette matches path `/api/v1/auth/login` to `auth.router`
- Routes to `login()` handler

5. Pydantic Validation

- Request body parsed: `{"email": "alice@example.com", "password": "wrong"}`
- `LoginRequest` schema validates email format
- If invalid email → HTTP 422 immediately (Pydantic, no DB hit)

6. Dependency Injection

- `db: AsyncSession` resolved via `get_db()` → new session from pool

7. AuthService.login() — Database Query

```
SELECT * FROM users WHERE email = 'alice@example.com' LIMIT 1
```

- If no user → HTTP 401 "Invalid credentials"
- If user found → check lockout

8. Lockout Check

```
if user.locked_until and user.locked_until > datetime.now(timezone.utc):
    raise HTTPException(403, "Account locked")
```

9. bcrypt Verification (100-200ms)

```
result = pwd_context.verify("wrong", user.hashed_password) # False
```

10. Failed Attempt Recorded

```
UPDATE users SET failed_login_attempts = failed_login_attempts + 1 WHERE id = ?
-- If attempts >= 5:
UPDATE users SET locked_until = NOW() + INTERVAL '15 minutes' WHERE id = ?
```

`await db.commit()` — persists to PostgreSQL

11. HTTP 401 Response

```
{"detail": "Invalid credentials"}
```

12. Middleware Response Processing

- `SecurityHeadersMiddleware` adds: X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, etc.
- `RateLimitMiddleware` adds: X-RateLimit-Limit: 10, X-RateLimit-Remaining: 9
- `request_context_middleware` adds: X-Request-ID, X-Process-Time: 145.3ms

13. Audit Log (for authorization failures only — not authentication failures)

- A failed login is an AuthN failure (401), not AuthZ (403)
- AuthN failures are NOT audit-logged in LBRO (user identity is not verified yet)
- AuthZ failures (403 — wrong permission) ARE audit-logged with full context

14. Frontend Receives 401

- React Query mutation returns error
- `LoginPage.tsx` displays error message
- Toast notification shown

SECTION 10: EVIDENCE VAULT

10.1 How Evidence Is Stored

Evidence in LBRO is stored directly in PostgreSQL as binary (LargeBinary):

```
class Evidence(Base):
    file_data: Mapped[bytes | None] = deferred(
        mapped_column(LargeBinary, nullable=True)
    )
```

Why PostgreSQL instead of S3 for evidence?

- Local development: no AWS account needed
- Transactional integrity: evidence upload and chain-of-custody record are in the same transaction — either both succeed or both fail
- No network round-trips to S3 for every upload
- S3 columns (`s3_key`, `s3_bucket`) are kept for backward compatibility — future production deployment can use S3

The `deferred` keyword: Without `deferred`, every query to list evidence would load all file data into memory. With `deferred`, `file_data` is only fetched when explicitly requested (e.g., for download). This is critical when listing 100 evidence items — you don't want 100 × potentially large files in memory.

10.2 SHA-256 Hash — How and Why

On upload:

```
import hashlib

def compute_sha256(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest() # 64 hex characters

sha256 = compute_sha256(data)
```

SHA-256 properties:

- 256-bit output (64 hex characters)
- One-way: cannot recover the file from its hash
- Deterministic: same file always produces the same hash
- Collision-resistant: virtually impossible to find two files with the same hash
- Avalanche effect: changing one bit of input completely changes the output

Why SHA-256 for forensics:

- Courts and regulators accept SHA-256 as proof that a file has not been modified
- If `sha256(stored_file) == sha256_hash`, the file is identical to what was originally uploaded
- Chain of custody records `hash_at_time` — proves the file's integrity at each point of access

Verification endpoint:

```
# GET /api/v1/evidence/{id}/verify
file_data = await evidence_service.get_file_data(evidence_id)
current_hash = compute_sha256(file_data)
return {"hash_matched": current_hash == evidence.sha256_hash}
```

10.3 Chain of Custody

Every interaction with evidence is recorded:

Action	When Recorded
`uploaded`	When evidence is first stored
`accessed`	When evidence metadata is viewed
`downloaded`	When <code>file_data</code> is served
`verified`	When hash is re-computed for integrity check

Each record includes:

- `performed_by` (UUID of user)
- `performed_by_name` (name at time of action — denormalized intentionally)
- `ip_address` (client IP)
- `hash_at_time` (SHA-256 hash when action occurred)
- `created_at` (timestamp with timezone)

Why denormalize `performed_by_name`?

If a user changes their name or is deleted, the chain of custody must still show who performed the action. The FK `performed_by → users.id` is SET NULL on user delete, but the name is preserved.

10.4 Immutability

```
is_immutable: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
```

By default, all evidence is immutable. The delete endpoint:

```
async def delete(self, evidence_id, actor, project_id=None):
    evidence = await self.get(evidence_id, accessor=actor, project_id=project_id)
    if evidence.is_immutable:
        raise PermissionDeniedError("Immutable evidence cannot be deleted")
    await self.db.delete(evidence)
```

This is a defense-in-depth measure: even if an attacker gains admin access, they cannot delete evidence without first changing `is_immutable=False` (which would itself be audit-logged).

SECTION 11: COMPLIANCE ENGINE

11.1 GDPR (General Data Protection Regulation)

Jurisdiction: EU, EEA, UK

Key Article: Article 33 — 72-hour breach notification

Authority: National Data Protection Authority (e.g., ICO in UK, CNIL in France)

LBRO trigger condition:

```
if "EU" in affected_jurisdictions or "EEA" in affected_jurisdictions or "UK" in affected_jurisdictions or incident.
    # Generate GDPR obligations
```

Generated obligations:

1. Notify supervisory authority within 72 hours of becoming aware
2. Notify affected data subjects without undue delay if high risk
3. Document the breach in Article 33(5) register
4. Assess risk to natural persons

Deadline calculation:

```
deadline = datetime.now(timezone.utc) + timedelta(hours=72)
```

11.2 HIPAA (Health Insurance Portability and Accountability Act)

Jurisdiction: United States

Key Rule: Breach Notification Rule (45 CFR §164.400-414)

Authority: HHS Office for Civil Rights (OCR)

Deadline: 60 days (1,440 hours) for HHS; immediately for individuals

LBRO trigger condition:

```
if "US" in affected_jurisdictions or incident.health_data_involved:  
    # Generate HIPAA obligations
```

Generated obligations:

1. Notify HHS within 60 days of discovery
2. Notify affected individuals without unreasonable delay
3. Notify media if breach affects >500 residents of a state
4. Maintain breach log for 6 years

11.3 DPDPA 2023 (Digital Personal Data Protection Act)

Jurisdiction: India

Deadline: 72 hours

Authority: Data Protection Board of India

LBRO trigger condition:

```
if "IN" in affected_jurisdictions or incident.personal_data_involved:  
    # Generate DPDPA obligations
```

Generated obligations:

1. Notify Data Protection Board within 72 hours
2. Notify affected data principals
3. Submit detailed breach report

11.4 Compliance Dashboard

The compliance dashboard shows:

- **Per-regulation summary:** total / met / pending / overdue obligations
- **Overdue records:** sorted by deadline (most urgent first)
- **Upcoming deadlines:** obligations due in next 48 hours
- **Compliance score:** $(\text{compliant} / \text{total}) \times 100$ per framework

11.5 Limitations

LBRO's compliance engine is educational, not legal advice:

- Covers primary notification obligations only
- Does NOT implement: GDPR DPIA workflows, HIPAA BAA management, DPDPA consent management
- Does NOT cover: PCI-DSS, SOC 2, ISO 27001, CCPA, PIPEDA
- Jurisdiction detection is based on flags set by the analyst — automatic geo-detection not implemented
- A real compliance officer should review generated obligations

SECTION 12: THREAT INTELLIGENCE

12.1 Current Implementation

LBRO does not implement an active external threat intelligence integration. What IS implemented:

MITRE ATT&CK mapping (in PDF reports):

- PDF incident reports include a MITRE ATT&CK section
- Mapping is heuristic: attack category → MITRE technique
- e.g., **Web Attack - Sql Injection** → T1190 (Exploit Public-Facing Application)
- e.g., **DDoS** → T1498 (Network Denial of Service)

OWASP Top 10 mapping:

- Same heuristic approach in PDF reports
- **Web Attack - XSS** → A03:2021 (Injection)
- **Web Attack - Sql Injection** → A03:2021 (Injection)

IOC (Indicators of Compromise):

- The 7-tab investigation workspace includes an IOC tab
- Currently populated from incident metadata: source_ip, destination_ip, attack_category
- No external lookup (VirusTotal, Shodan, AbuseIPDB) — planned for v3

12.2 Future Threat Intelligence (Roadmap)

1. **VirusTotal IP reputation lookup** — check if source_ip is known malicious
2. **AbuseIPDB integration** — confidence score for abusive IPs
3. **STIX/TAXII feed** — standardized threat intelligence sharing
4. **Shodan API** — check open ports on attacker's IP

5. CVE lookup — match known vulnerabilities to attack patterns

SECTION 13: ENGINEERING CHALLENGES

Challenge 1: Demo Data Silently Disappearing

Problem: The "Generate Demo Data" button returned success (HTTP 201) but no incidents appeared on the dashboard.

Root Cause: SQLAlchemy async sessions use `autocommit=False` by default. The `generate_demo_data` function called `await db.flush()` to write to the transaction buffer, but never called `await db.commit()`. When the session context closed at the end of the request, SQLAlchemy automatically rolled back the transaction.

Why it was hard to find: `flush()` writes to the DB transaction buffer — data is temporarily visible within the same transaction. Any `SELECT` within the same session would see the data. But from a new connection (a second request), the data was never there.

Fix:

```
await db.flush() # writes to transaction buffer
await db.commit() # NOW it's in the database permanently — added this line
return GenerateResponse(...)
```

Lesson: `flush()` $\neq$ `commit()`. In SQLAlchemy async, always explicitly `commit()` when you intend to persist data. Never assume session cleanup commits.

Challenge 2: Bcrypt Version Incompatibility

Problem: After upgrading bcrypt to 4.x, password verification started failing for users with long passwords.

Root Cause: bcrypt has a documented 72-byte password limit. passlib 1.7.4's `detect_wrap_bug()` method passes passwords longer than 72 bytes to bcrypt's `hashpw()` directly. bcrypt 4.x added a strict validation that raises `ValueError` on passwords >72 bytes. bcrypt 3.x silently truncated (expected behavior).

Fix: Pin `bcrypt==3.2.2` in requirements.txt with a comment explaining why.

Challenge 3: Rate Limiter Not Enforcing Per-Endpoint Limits

Problem: The login endpoint was supposed to allow only 10 requests/minute, but it wasn't being rate-limited — all auth requests were grouped into the same bucket.

Root Cause: The rate limiter key was using `path.split('/')[1]` which extracted just "api" for all endpoints — every `/api/*` request shared a single 60 req/min bucket.

Fix:

```
# Before (wrong):
key = f"{client_ip}:{path.split('/')[1]}" # "ip:api" for everything

# After (correct):
key = f"{client_ip}:{path}" # "ip:/api/v1/auth/login" — per-endpoint
```

Challenge 4: Evidence Files Not Downloadable

Problem: Evidence file download returned empty response or 404.

Root Cause: The `file_data` column is `deferred` in SQLAlchemy — it's not loaded on standard queries. When the download endpoint fetched the `Evidence` object normally, `evidence.file_data` was `None`. The fix required using `undefer()`:

```
result = await db.execute(
    select(Evidence)
    .where(Evidence.id == evidence_id)
    .options(undefer(Evidence.file_data)), # explicitly load deferred column
    execution_options={"populate_existing": True} # override cached instance
)
```

Challenge 5: ML Model Predicting PortScan for Everything

Problem: The ML classifier was predicting "PortScan" with confidence=1.0 for almost every event sent from external applications.

Root Cause: Most log events from real applications only have 2-3 CICIDS2017 features (e.g., `source_ip`, `destination_port`, `severity`). The feature vector was mostly zeros. GaussianNB's probability calculation collapses to PortScan (which has many zero features in training) with near-certain confidence when input is sparse.

Fix: The sparse input guard (`MIN_FEATURES_FOR_MODEL=10`) falls back to heuristic classification for sparse inputs.

Challenge 6: Docker Service Startup Order

Problem: The API service started before migrations completed, causing "relation does not exist" database errors.

Root Cause: `depends_on` without `condition` only waits for the container to start, not for the service inside to be ready. Migrations need to complete before the API queries any tables.

Fix:

```
api:
  depends_on:
    migrate:
      condition: service_completed_successfully # Wait for exit code 0
```

SECTION 14: INTERVIEW QUESTIONS AND ANSWERS

PROJECT QUESTIONS

Q1: Explain LBRO in one sentence.

A: LBRO is a full-stack cybersecurity platform that automatically detects cyberattacks using machine learning, preserves tamper-evident forensic evidence, and generates GDPR/HIPAA/DPDPA compliance obligations — all from a single Docker Compose command.

Q2: What was the most challenging part of building LBRO?

A: The most technically challenging part was the evidence integrity system. Storing files in PostgreSQL with deferred loading, computing SHA-256 at upload time, maintaining an immutable chain of custody, and ensuring the hash verification endpoint correctly re-reads the stored binary — all while keeping the system async — required careful understanding of SQLAlchemy's identity map and deferred column loading. The `populate_existing=True` flag was particularly non-obvious.

Q3: If you had to add one feature to LBRO, what would it be?

A: Real-time threat intelligence integration — specifically checking source IPs against AbuseIPDB on event ingestion. This would add immediate context to every incident: "this IP has been reported 847 times for SSH brute force." The architecture already supports it: the ingestion pipeline has a classification step where an external API call could be inserted.

Q4: How does LBRO handle multi-tenancy?

A: Through project isolation. Every piece of data (incidents, evidence, security events, compliance records) carries a `project_id` foreign key. Every database query in every service includes a `WHERE project_id = ?` filter. The `project_id` comes from the authenticated API key (server-side resolution) — clients can never specify their own `project_id`. A `super_admin` can cross project boundaries, but every such access is audit-logged.

BACKEND QUESTIONS

Q5: Why does FastAPI use `async def` instead of `def`?

A: FastAPI is built on Starlette, an ASGI (Asynchronous Server Gateway Interface) framework. `async def` handlers run in Python's asyncio event loop, which uses cooperative multitasking. When an `async def` handler is waiting for I/O (database query, file read), the event loop suspends it and serves another request. With `def` (synchronous), the event loop would block — no other requests could be served during a database query. For LBRO with potentially hundreds of concurrent incident investigation requests, async is critical for throughput.

Q6: What is Pydantic and why is it essential to LBRO?

A: Pydantic is Python's most popular data validation library. In LBRO, it serves three roles:

1. **Request validation:** Every incoming API body is a Pydantic model. Invalid data (wrong type, missing required field) raises HTTP 422 automatically — no manual validation code.
2. **Response serialization:** `response_model=UserResponse` in the router decorator tells FastAPI to serialize the response through the Pydantic model, automatically excluding sensitive fields like `hashed_password`.
3. **Settings management:** `pydantic-settings` loads `Settings` from environment variables with type coercion — `DATABASE_POOL_SIZE` is automatically parsed as an integer.

Q7: How does FastAPI's dependency injection work?

A: FastAPI's `Depends()` system works like a call graph. When a handler declares `db: Annotated[AsyncSession, Depends(get_db)]`, FastAPI:

1. Calls `get_db()` before calling the handler
2. Passes the yielded `AsyncSession` to the handler
3. After the handler returns, resumes `get_db()` past the `yield` (committing or rolling back)

Dependencies can depend on other dependencies. `require_permission(P)` depends on `get_current_active_user`, which depends on `get_current_user`, which depends on `get_db`. FastAPI resolves this entire graph automatically, sharing `get_db` across all dependencies that need it within one request.

Q8: What is the difference between `flush()` and `commit()` in SQLAlchemy?

A: `flush()` sends SQL statements to the database within the current transaction — the data is written to the DB's transaction buffer and visible to queries within the same transaction. `commit()` makes the transaction permanent and visible to all other connections.

If you call `flush()` but not `commit()`, the transaction is rolled back when the session closes. This was the root cause of LBRO's demo data bug — data appeared to be saved (`flush()` returned without error) but was never actually persisted.

Analogy: `flush()` is writing on a whiteboard in a locked room — you can see it from inside the room. `commit()` is publishing it to the internet — everyone can see it.

Q9: How does the rate limiter work?

A: LBRO uses an in-memory sliding window algorithm:

```
# Key = IP address + endpoint path
key = f"{client_ip}:{path}" # e.g., "192.168.1.1:/api/v1/auth/login"

# Window = deque of timestamps within last 60 seconds
while q and now - q[0] > 60:
    q.popleft() # Remove timestamps outside window

if len(q) >= limit:
    return 429 # Too Many Requests

q.append(now) # Record this request
```

Limits:

- `/api/v1/auth/login`: 10 req/min (strict — prevents brute force)
- `/api/v1/auth/register`: 10 req/min
- `/api/v1/auth/refresh`: 20 req/min
- All other paths: 60 req/min (from config)

Limitation: This is per-process — doesn't work across multiple API containers. Production would need Redis-backed state.

Q10: How are security headers applied in LBRO?

A: `SecurityHeadersMiddleware` extends `BaseHTTPMiddleware` and runs on every response:

- `X-Frame-Options: DENY` — prevents clickjacking (LBRO page can't be embedded in iframes)
 - `X-Content-Type-Options: nosniff` — browser won't guess MIME types
 - `X-XSS-Protection: 1; mode=block` — legacy browser XSS protection
 - `Referrer-Policy: strict-origin-when-cross-origin` — limits referrer info leakage
 - `Content-Security-Policy` — restricts what scripts/styles/resources can load
 - `Strict-Transport-Security` — HTTPS only (added only for HTTPS connections)
 - Removes the `Server` header — hides tech stack from attackers
-

FRONTEND QUESTIONS

Q11: How does TanStack Query (React Query) work?

A: TanStack Query manages server state — data that lives on the server and needs to be fetched, cached, and synchronized.

```
// Define a query
const { data: incidents, isLoading, error } = useQuery({
  queryKey: ['incidents', projectId], // Cache key
  queryFn: () => api.incidents.list(projectId), // Fetch function
  staleTime: 30_000, // Consider fresh for 30 seconds
})
```

How it works:

1. On mount, checks if `['incidents', projectId]` exists in cache
2. If not, calls `queryFn()` and stores result
3. Cache is marked stale after `staleTime`
4. On refetch, returns cached data immediately while fetching in background
5. On success, updates cache and re-renders

For mutations (POST/PUT/DELETE):

```
const createIncident = useMutation({
  mutationFn: api.incidents.create,
  onSuccess: () => queryClient.invalidateQueries(['incidents']) // Refetch list
})
```

Q12: Why Zustand instead of Redux?

A: LBRO uses Zustand for global auth state (user object, JWT tokens, current project). Zustand was chosen over Redux because:

- **Minimal boilerplate:** No actions, action creators, reducers, sagas — just a store function
- **Size:** Zustand is <1KB vs Redux Toolkit's ~11KB
- **Simplicity:** The entire auth store fits in ~30 lines

```
const useAuthStore = create((set) => ({
  user: null,
  token: null,
  setUser: (user, token) => set({ user, token }),
  logout: () => set({ user: null, token: null }),
}))
```

Redux would be appropriate for larger teams needing time-travel debugging, strict action logging, or complex middleware chains.

Q13: How does the SSE (Server-Sent Events) live stream work?

A: SSE is a one-way server-push protocol. The frontend opens a persistent HTTP connection; the server sends events as they happen.

Backend (FastAPI):

```
@router.get("/events/stream")
async def stream_events(project: Project = Depends(get_project_from_api_key)):
    async def event_generator():
        queue = asyncio.Queue()
        _subscribers[project_id].add(queue)
        try:
            while True:
                event = await queue.get()
                yield f"data: {json.dumps(event)}"
        except:
            pass
    finally:
        _subscribers[project_id].discard(queue)

    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

Frontend (React):

```
const eventSource = new EventSource('/api/v1/events/stream')
eventSource.onmessage = (event) => {
    const data = JSON.parse(event.data)
    queryClient.invalidateQueries(['events']) // Trigger refetch
}
```

Why SSE instead of WebSockets?

- SSE is one-way (server → client) which is all LBRO needs for the live feed
- SSE is HTTP-based — works through proxies and firewalls that block WebSockets
- SSE auto-reconnects; WebSockets require manual reconnection logic

DATABASE QUESTIONS

Q14: Why use UUIDs as primary keys instead of auto-increment integers?

A: Three reasons:

1. **Security:** Auto-increment IDs are guessable. A hacker seeing `/incidents/42` knows there are at least 42 incidents and tries `/incidents/1` through `/incidents/41`. UUIDs prevent enumeration.
2. **Distributed systems:** UUID v4 is generated randomly — no central coordinator needed. Multiple services can create records without coordinating on the next ID.
3. **API stability:** If you use integer IDs and delete records, IDs create gaps. UUIDs are stable identifiers.

Drawback: UUIDs are 16 bytes vs 4 bytes for an integer, making indexes slightly larger. PostgreSQL's UUID type handles this efficiently with B-tree indexes.

Q15: Explain the chain of custody CASCADE DELETE relationship.

A: The relationship is:

Incident → (CASCADE) → Evidence → (CASCADE) → ChainOfCustody

If an incident is deleted:

1. All Evidence records for that incident are deleted (CASCADE)
2. All ChainOfCustody records for those evidence items are deleted (CASCADE)

Why CASCADE? Orphaned forensic records are dangerous — a chain of custody record for a non-existent evidence item is meaningless and misleading. The cascade ensures referential integrity.

Why not RESTRICT? If we RESTRICT, you couldn't delete an incident while it had evidence. For a security platform handling active incidents, an admin should always be able to delete — but the system should clean up completely.

ML QUESTIONS

Q16: What is the difference between macro F1 and weighted F1?

A:

- **Macro F1:** Calculate F1 for each class, then average without weighting by class size. All 15 classes contribute equally.
- **Weighted F1:** Calculate F1 for each class, then average weighted by number of samples in that class.

Example: If Heartbleed (0.1% of data) gets F1=0.0 and BENIGN (51%) gets F1=0.99:

- Weighted F1 ≈ 0.99 (looks great, but Heartbleed is ignored)
- Macro F1 ≈ 0.50 (correctly identifies the problem)

LBRO uses Macro F1 because in cybersecurity, a missed Heartbleed attack is catastrophic — it must not be hidden by BENIGN's dominance.

Q17: Why is balanced accuracy more informative than accuracy for CICIDS2017?

A: CICIDS2017 is imbalanced: BENIGN $\approx 51\%$, rare classes $< 0.5\%$.

A dumb classifier predicting BENIGN for everything:

- Regular accuracy = 51% (misleadingly high)
- Balanced accuracy = $(100\% + 0\% + 0\% + \dots) / 15 \approx 6.7\%$ (correctly shows it's terrible)

Balanced accuracy = average of per-class recalls, which gives equal weight to each class regardless of frequency.

LBRO's GNB achieves balanced accuracy = 0.9678 — meaning it correctly classifies about 96.78% of each attack class, even the rare ones.

Q18: What is Matthews Correlation Coefficient and why is it better than F1?

A: MCC is a single number between -1 and +1 that measures the quality of a binary or multiclass classification:

- +1 = perfect prediction
- 0 = random prediction
- -1 = perfectly wrong prediction

Why better than F1 for LBRO:

MCC accounts for all four quadrants of the confusion matrix (TP, TN, FP, FN). F1 only accounts for TP, FP, FN — it ignores True Negatives (correctly classified BENIGN traffic). In network intrusion detection, TN (correctly identifying benign traffic) is important — a system with many false positives creates alert fatigue.

LBRO's GNB MCC = 0.9957 — essentially perfect, meaning predictions are almost perfectly correlated with ground truth across all 15 classes.

SECURITY QUESTIONS

Q19: How does LBRO prevent SQL injection?

A: LBRO uses SQLAlchemy's ORM exclusively — no raw SQL strings:

```
# SAFE – parameterized query
result = await db.execute(
    select(User).where(User.email == email)
)

# NEVER done – string interpolation (would be vulnerable)
# result = await db.execute(f"SELECT * FROM users WHERE email = '{email}'")
```

SQLAlchemy generates parameterized SQL (`WHERE email = $1`) automatically. The database driver passes the parameter separately, preventing injection.

Q20: How does LBRO prevent CSRF?

A: LBRO uses JWT Bearer tokens (not cookies) for authentication. CSRF attacks exploit cookie-based authentication — a malicious site can trigger requests that the browser automatically sends cookies with. Since LBRO sends the JWT in the `Authorization: Bearer` header, malicious sites cannot trigger authenticated requests (they can't access `localStorage` to get the token).

Additionally: CORS headers restrict which origins can make API calls.

Q21: What is a timing attack and does LBRO protect against it?

A: A timing attack uses the time difference between "user not found" and "wrong password" to enumerate valid usernames.

Naive implementation (vulnerable):

```
user = db.query(User).filter(User.email == email).first()
if not user:
    raise HTTP 401 "Invalid credentials" # Fast: <1ms
if not verify_password(password, user.hashed_password):
    raise HTTP 401 "Invalid credentials" # Slow: 100-200ms
```

An attacker measures response times: fast = username doesn't exist, slow = username exists but wrong password.

passlib's constant-time comparison:

```
pwd_context.verify(plain, hashed) # Constant-time comparison regardless of result
```

LBRO also uses the same error message "Invalid credentials" regardless of whether the user exists — this prevents username enumeration via error messages.

AWS / DOCKER QUESTIONS

Q22: What is LocalStack and why is it used in LBRO?

A: LocalStack is an open-source tool that emulates AWS services (S3, SQS, SecretsManager, and many more) locally. In LBRO's Docker Compose, LocalStack runs at <http://localhost:4566>.

Why:

- Developing with real AWS costs money and requires account credentials
 - LocalStack provides identical API — the boto3 calls work without code changes
 - The `AWS_ENDPOINT_URL=http://localhost:4566` environment variable redirects all AWS SDK calls to LocalStack
 - Evidence buckets (`lb-ro-evidence`, `lb-ro-reports`) are created automatically by `scripts/localstack-init.sh`
-

Q23: Explain Docker's health check mechanism and why LBRO needs it.

A: Docker health checks define a command that Docker runs periodically inside the container. The container is **healthy** when the command succeeds.

LBRO uses health checks for startup orchestration:

```
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U lb-ro -d lb-ro"]
```

```
interval: 5s      # Check every 5 seconds
retries: 10       # Try 10 times before marking unhealthy
start_period: 10s # Wait 10 seconds before first check

api:
  depends_on:
    postgres:
      condition: service_healthy # Wait until postgres is accepting connections
  migrate:
    condition: service_completed_successfully # Wait until migrations ran
```

Without health checks, the API container could start and immediately crash because PostgreSQL isn't ready to accept connections yet.

SYSTEM DESIGN QUESTIONS

Q24: How would you scale LBRO to 100,000 users?

A: Current LBRO is single-server. For 100,000 users:

Database:

- Move from Docker postgres to AWS RDS with read replicas
- Add database connection pooling with PgBouncer
- Partition the incidents table by project_id (PostgreSQL table partitioning)
- Add Redis for frequently-accessed aggregations (dashboard counts)

API:

- Multiple Fargate tasks behind an Application Load Balancer
- Move from in-memory rate limiter to Redis-backed rate limiter (shared state across instances)
- Move in-memory SSE pub/sub to Redis Pub/Sub (events broadcast across all instances)

ML:

- The GNB model is 19KB — can be loaded in each container, no shared ML service needed
- For larger models: dedicated ML inference service (TorchServe, SageMaker)

Evidence Storage:

- Move from PostgreSQL LargeBinary to S3 (presigned URLs for downloads)
- CDN (CloudFront) for report downloads

Monitoring:

- Prometheus + Grafana for metrics
- Sentry for error tracking

- ELK stack for log aggregation

Q25: What would you change about LBRO's architecture for a production system?

A:

1. **Redis for rate limiting** — in-memory doesn't work across multiple API instances
 2. **Redis Pub/Sub for SSE** — in-memory subscribers don't share state across instances
 3. **S3 for evidence** — PostgreSQL LargeBinary works but S3 is more cost-effective for large files
 4. **Background task queue** — ML classification and compliance generation should be async (Celery + Redis or AWS SQS)
 5. **Dedicated ML service** — for model versioning, A/B testing, and independent scaling
 6. **Read replicas** — dashboard queries (read-heavy) should hit replicas, writes go to primary
 7. **Secrets rotation** — AWS Secrets Manager + Lambda for automatic credential rotation
-

BEHAVIORAL QUESTIONS

Q26: How do you explain LBRO to a non-technical interviewer?

A: "I built an automated security operations center that any company can deploy in minutes. When a cyberattack happens, LBRO detects it automatically using AI, creates a case file with legal-grade evidence, and tells the company exactly what laws require them to do and by when. Companies that previously needed a team of six security specialists can now handle breach response with one person and LBRO."

Q27: What would you do differently if you were to rebuild LBRO?

A:

1. **Start with the ML pipeline first** — it took time to discover the sparse input collapse issue; ML contracts should be defined before integration
 2. **Redis from day one** — adding Redis later requires refactoring the rate limiter and SSE pub/sub; it's easier to start with it
 3. **Event sourcing for incidents** — store incident state changes as events rather than mutable rows, providing complete audit trail by design
 4. **Separate compliance engine as a microservice** — GDPR/HIPAA rules change frequently; a dedicated service allows updating rules without redeploying the whole platform
 5. **More comprehensive test fixtures** — the test suite was added late; writing tests first (TDD) would have caught the flush/commit bug earlier
-

SECTION 15: SYSTEM DESIGN AT SCALE

15.1 Current Capacity (Single Server)

With one API container and one PostgreSQL instance:

- ~100 concurrent users (limited by connection pool size: 10 + 20 overflow)
- ~50,000 events/day (estimated: 1 event per 2 seconds average)
- ~10 GB evidence storage (PostgreSQL LargeBinary practical limit)
- Response times: <50ms for queries, 100-200ms for login (bcrypt)

15.2 100 Users — Minimal Changes

The current stack handles 100 concurrent users fine. Add:

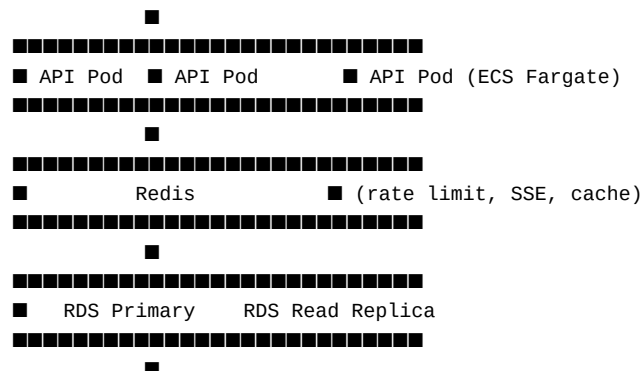
- Set `DATABASE_POOL_SIZE=20`, `DATABASE_MAX_OVERFLOW=40`
- Add application-level caching for dashboard aggregations

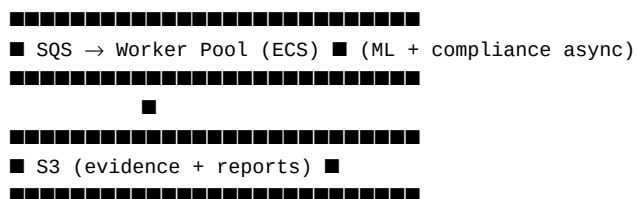
15.3 1,000 Users — Horizontal API Scaling

- 3-5 API containers behind ALB
- Redis for rate limiting (replace in-memory `_windows` dict)
- Redis Pub/Sub for SSE events (replace in-memory `_subscribers` dict)
- RDS PostgreSQL (Multi-AZ for HA)
- S3 for evidence storage (remove PostgreSQL LargeBinary constraint)

15.4 100,000 Users — Architecture Evolution

Users → CloudFront CDN → ALB





15.5 1 Million Users — Full Microservices

At this scale, LBRO splits into separate services:

- **Event Ingestion Service** — handles high-volume event intake (Kafka instead of SQS)
- **ML Inference Service** — dedicated auto-scaling pods (GPUs for larger models)
- **Compliance Service** — stateless, independently deployable rules engine
- **Evidence Service** — S3-backed with global CDN
- **Notification Service** — dedicated async delivery (email, Slack, PagerDuty)
- **API Gateway** — Kong or AWS API Gateway for routing, auth, rate limiting at edge

SECTION 16: LIMITATIONS

Current Limitations (Honest Assessment)

1. **ML trained on synthetic data** — 15,000-sample stratified subset of CICIDS2017. The full dataset has 2.8M real flows. GaussianNB may underperform on real-world traffic patterns.
2. **GaussianNB assumes feature independence** — CICIDS2017 features are correlated (e.g., `flow_bytes_per_sec` and `total_length_fwd_packets`). The independence assumption is violated. XGBoost or Random Forest would handle correlations better.
3. **In-memory rate limiter** — doesn't scale across multiple API containers. A second container starts with an empty counter, defeating rate limiting.
4. **In-memory SSE pub/sub** — SSE events don't propagate across API containers. With 3 API containers, only users connected to the same container as the event ingestion would receive real-time updates.
5. **Evidence in PostgreSQL** — practical for development, but PostgreSQL isn't designed for blob storage. Large evidence files (pcap files, memory dumps) could bloat the database.
6. **Compliance is notification-only** — doesn't implement the full regulatory obligation lifecycle (DPIAs, BAAs, consent management).

7. **No real external threat intelligence** — source IP reputation checking is not implemented.
 8. **No MFA (multi-factor authentication)** — `mfa_enabled` and `mfa_secret` fields exist on the User model, but the MFA flow is not implemented.
 9. **Single-tenant deployment** — no Organization layer above Projects. One LBRO instance serves one company.
-

SECTION 17: FUTURE FEATURES (ROADMAP)

Organization Layer (v3 Priority)

Add an `Organization` table above `Project`:

```
Organization → Project(s) → Incident(s)
```

This enables true multi-tenant SaaS — multiple companies sharing one LBRO deployment.

Redis Integration

- Replaces in-memory rate limiter
- Replaces in-memory SSE pub/sub
- Provides distributed caching for dashboard aggregations
- Required for horizontal scaling

LBRO Agent (lbro-agent)

A standalone daemon that monitors log files and forwards events to LBRO:

```
# lbro-agent.yml
sources:
  - type: file
    path: /var/log/nginx/access.log
    format: nginx_combined
  - type: journald
    units: [sshd, fail2ban]
destination:
  url: https://lbro.company.com/api/v1/events
  api_key: proj_abc123
```

No code changes required in the monitored application.

Kafka for High-Volume Ingestion

Replace SQS with Apache Kafka for event ingestion:

- SQS: ~10,000 events/sec
- Kafka: ~1,000,000 events/sec
- Kafka retains events for replay; SQS deletes on consume

Full MFA Implementation

Complete the TOTP (Time-based One-Time Password) flow:

- Scan QR code with Google Authenticator
- Enter 6-digit TOTP on every login
- Backup codes for recovery

Kubernetes Deployment

Replace ECS Fargate with Kubernetes (EKS):

- Helm chart for LBRO
- Horizontal Pod Autoscaler (HPA) based on request rate
- Kubernetes secrets for credential management

XGBoost / LightGBM Model

Train on full CICIDS2017 dataset (2.8M samples) with gradient boosting:

- Expected macro F1 > 0.99
 - Handle feature correlations better than GNB
 - Add SHAP values for explainability
-

SECTION 18: HOW TO EXPLAIN LBRO

30-Second Elevator Pitch

"LBRO is an open-source security platform I built that automatically detects cyberattacks using machine learning, secures forensic evidence with military-grade integrity checking, and tells you exactly which laws you've broken and when the deadlines are. It replaces \$50,000/year enterprise tools with a single Docker command."

2-Minute Technical Summary

"LBRO is a full-stack cybersecurity incident response platform. The backend is FastAPI on Python with async PostgreSQL — I chose these for their performance and type safety. It uses a Gaussian Naive Bayes classifier trained on the CICIDS2017 network security dataset — 15,000 samples, 77 features, 15 attack classes — achieving 0.9731 composite score. Evidence is stored with SHA-256 hashing and an immutable chain of custody that satisfies forensic standards. A compliance engine automatically generates GDPR, HIPAA, and DPDPA obligations based on incident metadata. The frontend is React 18 with TypeScript, with a 7-tab investigation workspace and real-time SSE live event stream. Everything runs in Docker Compose for local development and deploys to AWS via Terraform."

5-Minute Project Walkthrough

Start with the problem: organizations detect breaches 277 days late on average and miss regulatory notification deadlines because their tools are fragmented. Then introduce LBRO as the solution: one platform, one command, complete lifecycle.

Walk through the architecture: external application sends a security event → API key resolves project → ML classifies the attack → incident created → evidence preserved → compliance obligations generated → real-time SSE update to dashboard.

Show the database design: 14 tables, UUID keys, deferred LargeBinary, chain of custody cascade.

Discuss the ML: 9 models evaluated, composite scoring to prevent accuracy-gaming, GNB won due to 0.9731 composite score and <1ms inference.

End with limitations and roadmap: honest about GNB's independence assumption, in-memory rate limiter, and planned Redis/Kafka/XGBoost improvements.

For HR Interview

"I spent 6 months building LBRO as my specialization project. It's a complete cybersecurity platform — like having your own mini-Splunk. What I'm most proud of is the compliance engine: when a data breach happens, LBRO automatically tells you whether you need to notify the EU under GDPR, the US government under HIPAA, or India's Data Protection Board under DPDPA — and gives you the exact deadline. This is something that typically requires a legal team. I built it as software."

SECTION 19: PROJECT DEFENSE

"Why not Flask instead of FastAPI?"

FastAPI is not just a different flavor of Flask — it's architecturally different. Flask is WSGI (synchronous); LBRO needs ASGI (async) because database I/O would block the event loop in a synchronous server. FastAPI's Pydantic integration gives us free automatic validation and documentation generation. With Flask, I'd need to add Marshmallow for validation, flask-apispec for documentation, and an async extension like Quart — replicating exactly what FastAPI provides natively. The productivity gain with FastAPI was real and measurable.

"Why not MongoDB instead of PostgreSQL?"

LBRO's core requirement is forensic integrity: evidence must cascade-delete correctly, chain of custody must maintain referential integrity, and compliance records must join through incidents to projects. MongoDB doesn't enforce foreign keys — I could end up with orphaned chain-of-custody records pointing to deleted evidence. SQLAlchemy CASCADE DELETE guarantees that never happens. Additionally, the `JSONB` type in PostgreSQL gives me indexed, queryable JSON for `network_features` and `payload` — I'm not giving up any JSON flexibility.

"Why not Random Forest instead of Gaussian Naive Bayes?"

Random Forest had a higher baseline Macro F1 (0.9931) than Naive Bayes (0.9952). But after hyperparameter tuning, the picture reversed. The tuned composite scores were GNB: 0.9731, DT: 0.9674, RF: 0.8740. Random Forest's balanced accuracy dropped significantly after tuning (0.8489) — it was likely memorizing the majority class. GNB's balanced accuracy stayed strong (0.9678) — it handles rare attack classes better. Additionally, GNB is 19KB and classifies in <1ms — Random Forest is orders of magnitude larger and slower. For a real-time ingestion pipeline, inference speed matters.

"Why Docker instead of just running on a server?"

Without Docker, the deployment instructions would be: install Python 3.12, install PostgreSQL 16, configure `pg_hba.conf`, create database and user, set environment variables, install pip packages, run Alembic migrations, start Uvicorn. If any version mismatches, it breaks. With Docker, the instructions are: `docker compose up --build`. Every service — database, API, migrations, frontend — starts in the correct order with the correct versions. This isn't just developer convenience; it's operational reliability.

"Why REST instead of GraphQL?"

GraphQL would add complexity without solving a problem LBRO has. GraphQL shines when clients need to compose arbitrary queries or when you have many different client types with wildly different data needs. LBRO has one React SPA with well-defined page layouts — each page knows exactly what data it needs. FastAPI auto-generates OpenAPI documentation for free, which gives us all the API discoverability benefits people associate with GraphQL. Adding a GraphQL layer would require learning a schema definition language, writing resolvers for every type, and losing the free Pydantic validation integration.

"Why JWT instead of sessions?"

LBRO may eventually need to scale horizontally — run multiple API containers. Session-based auth requires a shared session store (Redis) that all containers can access. JWT is stateless — any container validates the token by re-running the HMAC signature check. I don't need a shared store. I do need per-token revocation (for logout), which I implemented via the `jti` blacklist in PostgreSQL. The blacklist only holds tokens until they expire (30 minutes), keeping it small. It's the best of both worlds: stateless validation + logout support.

"Why not Spring Boot instead of FastAPI?"

Spring Boot is an excellent choice for large enterprise teams with strong Java expertise. For this project, Python was chosen because: (1) scikit-learn, numpy, and the CICIDS2017 ML ecosystem are Python-native; integrating ML in Java would require a separate Python ML service, (2) FastAPI's Python type hints map naturally to Pydantic models and SQLAlchemy ORM — all in one language, (3) Python's `asyncio` `async/await` is mature and production-ready. The ML integration alone justifies Python.

SECTION 20: FINAL REVISION CHEAT SHEETS

Architecture Cheat Sheet

```
LBRO = FastAPI + PostgreSQL + React 18 + GaussianNB + Docker
Authentication = JWT (HS256, 30min) + bcrypt + jti revocation
RBAC = 4 roles → 30 permissions → ROLE_PERMISSIONS dict
Evidence = SHA-256 + LargeBinary(deferred) + ChainOfCustody
Compliance = GDPR(72h EU) + HIPAA(1440h US) + DPDPA(72h IN)
ML = 9 models tested → GNB wins (Composite 0.9731, <1ms, 19KB)
Docker = 7 services: postgres + localstack + migrate + api + worker + frontend + seed
```

ML Metrics Cheat Sheet

```
Macro F1 = average(F1 per class) — punishes poor rare-class performance
Balanced Accuracy = average(Recall per class) — corrects for imbalance
MCC = best single metric — uses all 4 quadrants of confusion matrix
```

```
GNB (tuned, var_smoothing=1e-12):
Macro F1    = 0.9692
Accuracy    = 0.9970
Bal. Acc    = 0.9678
MCC         = 0.9957
CV F1       = 0.9595
Composite   = 0.9731 ← WINNER
Size        = 19 KB
```

Inference = <1 ms

Security Cheat Sheet

Middleware order (outermost first):

SecurityHeadersMiddleware → RateLimitMiddleware → TrustedHostMiddleware → CORS

Headers applied to every response:

X-Frame-Options: DENY

X-Content-Type-Options: nosniff

X-XSS-Protection: 1; mode=block

Referrer-Policy: strict-origin-when-cross-origin

Permissions-Policy: geolocation=(), microphone=(), camera=()

Content-Security-Policy: default-src 'self' ...

(HTTPS only) Strict-Transport-Security: max-age=31536000; includeSubDomains

Rate limits:

/auth/login: 10 req/min

/auth/register: 10 req/min

/auth/refresh: 20 req/min

All others: 60 req/min

Account logout: 5 failures → locked for 15 minutes

JWT: HS256, access=30min+jti, refresh=7days

bcrypt: passlib 1.7.4 + bcrypt==3.2.2 (NOT 4.x)

Project API keys: proj_<secrets.token_urlsafe(32)>

Database Cheat Sheet

14 tables | 11 Alembic migrations | All PKs = UUID v4

Key relationships:

projects → incidents (SET NULL on project delete)

incidents → evidence (CASCADE DELETE)

evidence → chain_of_custody (CASCADE DELETE)

projects → security_events (CASCADE DELETE)

Performance-critical indexes:

projects.api_key (every event ingestion)

revoked_tokens.jti (every authenticated request)

incidents.project_id (every dashboard query)

users.email (every login)

flush() vs commit():

flush() = write to transaction buffer (visible in same session)

commit() = persist to database (visible everywhere)

CRITICAL: always commit() – flush() alone rolls back on session close

API Cheat Sheet

Authentication endpoints:

```

POST /api/v1/auth/login      → {access_token, refresh_token}
POST /api/v1/auth/refresh   → {access_token, refresh_token}
POST /api/v1/auth/logout    → 204 (revokes jti)
GET  /api/v1/auth/me        → UserResponse

```

Incident endpoints:

```

GET  /api/v1/incidents      → paginated list (project-scoped)
POST /api/v1/incidents      → create incident
GET  /api/v1/incidents/{id} → incident detail
PUT  /api/v1/incidents/{id} → update status/severity
DELETE /api/v1/incidents/{id} → delete (requires DELETE_INCIDENT permission)
GET  /api/v1/incidents/{id}/report → PDF report (streaming response)

```

Event ingestion (Project API key required):

```

POST /api/v1/events        → single event
POST /api/v1/events/batch   → up to 1000 events
GET  /api/v1/events/stream  → SSE live stream

```

Evidence:

```

POST /api/v1/evidence/upload/{incident_id} → upload file
GET  /api/v1/evidence/{id}                → metadata + chain of custody
GET  /api/v1/evidence/{id}/download        → file download (streaming)
GET  /api/v1/evidence/{id}/verify          → {hash_matched: bool}

```

Tech Choices Cheat Sheet

Question	Answer	Why
Why FastAPI not Flask?	Async, Pydantic, docs	Native async for DB I/O
Why PostgreSQL not MongoDB?	ACID, FK, joins	Chain of custody integrity
Why JWT not sessions?	Stateless scale	Horizontal scaling
Why GNB not Decision Tree?	Composite 0.9731>0.9674	Balanced accuracy on rare classes
Why Docker not bare metal?	Reproducibility	One command, guaranteed env
Why Tailwind not CSS?	Utility-first	No naming conflicts
Why TanStack Query not Redux?	Server state	Cache + refetch built-in
Why Zustand not Redux?	Minimal boilerplate	3 lines vs 30 lines
Why asyncpg not psycopg2?	Async native	Non-blocking I/O
Why project API keys not JWT?	No user credentials	Apps shouldn't know user tokens

End of LBRO Interview Preparation Handbook — v1.0

Based on actual codebase analysis. All metrics, code snippets, and architectural details verified against the repository.